Auth Service and Security Implementation Guide

Decision

Implement authentication and token issuance in **Auth Service**, validate access tokens at **API Gateway and every protected service**, and enforce account and transaction permissions in the service that owns the data. Gateway checks reject unwanted traffic early; they cannot replace service checks. This follows the project brief's Auth Service and gateway responsibilities and the OWASP microservices security guidance.

This repository is a learning project. The design below is a practical target for it, not a claim that security is already implemented. As inspected on 7 October 2026, `auth-service`, `api-gateway`, `account-service`, `approval-service`, `ledger-service`, `notification-service`, and `reconciliation-service` contain only generated Spring Boot applications. They use Java 21 and Spring Boot 3.5.16 and have no security dependencies, controllers, or databases configured. The current `compose.yaml` provides a transaction PostgreSQL database and Kafka only. `transaction-service` is absent from the current checkout; `docs/transaction-service-reference.md` describes an older implementation. Add that service or restore it before applying its security rules.

Responsibility by component

Component	Implement here	Do not rely on it for
Auth Service	User registration, password authentication, account status, role assignment, OAuth2 clients and token issuance, signing key management, refresh/revocation, authentication audit	Deciding whether a customer owns a particular account or whether a checker may approve a particular request
API Gateway	Validate external access tokens, route only intended public paths, rate limit, TLS termination, correlation IDs, reject spoofed identity headers	Being the sole place that protects downstream services
Account Service	Validate token and intended audience, check customer ownership and account status before reads or changes; authenticate internal operation callers	Trusting a `customerId` supplied in a request body
Transaction Service, when restored	Validate token, check source-account ownership and transaction visibility, enforce transaction scopes and idempotency	Assuming a token alone authorizes movement from an account
Approval Service	Validate token, require maker/checker permissions, compare authenticated checker with stored maker, enforce approval state atomically	Trusting a checker ID supplied by the client
Ledger, Notification, Reconciliation	Protect management/read APIs and internal calls; authenticate Kafka producers/consumers and authorize topic access	Treating an event's `actorId` as proof of the current HTTP caller's identity

```
Client -- access token --> API Gateway -- access token --> Protected API
                                |                          |
                          validate JWT               validate JWT
                                |                          |
Auth Service -- public JWKs --+--------------------------+
       |
  users, clients, grants and signing keys in Auth DB
```

Keep service ports private to the Docker network or internal network. A client should normally reach business APIs through the gateway. Still protect direct service endpoints because test traffic, a routing error, or a compromised internal caller can bypass gateway policy. Use HTTPS outside localhost. Do not accept a caller-supplied `X-User-Id`, `X-Role`, or similar header as authentication.

Recommended token model

Use **Spring Authorization Server** in Auth Service rather than writing JWT signing, refresh, and OAuth2 grant logic from scratch. It supports authorization code with PKCE for user-facing clients, refresh tokens, client credentials for service identities, a token endpoint, revocation, discovery, and a JWK Set endpoint. For the current Spring Boot 3.5.x project, use dependencies managed by its parent/BOM and the compatible Spring Security 6.5 documentation. The Spring Authorization Server reference describes these components.

For a browser or mobile client, register an OAuth2 client and use **authorization code with PKCE**. The human enters credentials at Auth Service's login page. The client exchanges the code at `/oauth2/token`; the password is never sent to Account or Transaction Service. For a trusted backend-to-backend call, register a separate confidential client and use **client credentials** with only the needed scope. Do not use the old OAuth password grant or issue a service token to a browser.

Start with signed JWT access tokens and a shared `aud` value of `banking-api` for the local exercise. Configure both the gateway and each HTTP resource service to accept only that audience and the Auth Service issuer. This is a deliberate first-phase simplification; if services later need separate audiences, use token exchange or separate tokens, then reject a token intended for another service. Never use an OIDC ID token as an API access token. Spring's resource-server guide documents issuer, signature, expiry, and audience validation; OWASP identity propagation guidance explains the boundary.

Suggested access-token claims:

Claim	Value / use
`iss`	Stable Auth Service issuer URL; same configured value in all environments that validate it
`sub`	Immutable user UUID for user tokens, or client ID for service tokens
`aud`	`banking-api` in phase 1; reject a missing or different audience
`exp`, `iat`, `nbf`	Short validity; use 5-15 minutes as an initial exercise setting and adjust to the revocation requirement
`jti`	Unique token ID for audit and optional emergency denylist
`scope`	Operations the OAuth2 client and principal may perform, such as `accounts.read`, `transactions.create`, `approvals.decide`
`roles`	`CUSTOMER`, `MAKER`, `CHECKER`, `ADMIN` for user tokens only; never accept a role from registration input

Use an asymmetric signing key (for example RSA) held only by Auth Service. Publish public keys through its JWK Set endpoint. Persist the private key outside source control; load it from a secret store or protected local file, and rotate keys with an overlap period while old access tokens expire. Never generate a new production signing key on each restart. Do not put passwords, account numbers, balances, or other sensitive banking data in JWT claims. Keep refresh tokens server-side through the authorization persistence layer, revoke them on logout/password reset/disablement, and rotate them where the client configuration supports it. A locally validated JWT can remain usable until expiry after logout; if immediate invalidation is required, add a checked revocation mechanism or use opaque tokens/introspection. OWASP JWT guidance covers that trade-off.

OAuth2 client scopes and a human user's roles are different controls. A client being registered for `approvals.decide` must never make every logged-in customer a checker. Restrict issued user scopes by the user's allowed role/policy, and always require the matching role plus the Approval Service's maker/checker data checks. Similarly, a client-credentials token represents a service, not a customer; do not give it customer ownership privileges.

Build Auth Service in this repository

1. **Add dependencies** in `auth-service/pom.xml`: `spring-boot-starter-web`, `spring-boot-starter-security`, `spring-boot-starter-oauth2-authorization-server`, `spring-boot-starter-data-jpa` or JDBC, `spring-boot-starter-validation`, `postgresql`, and `spring-boot-starter-test`. Add Flyway or Liquibase for versioned schema migrations. Let Spring Boot manage starter versions; do not mix in old `spring-security-oauth2` or `@EnableAuthorizationServer` tutorials.
2. **Give Auth Service its own PostgreSQL database.** Add `auth-db` to `compose.yaml` with a distinct user/database, a named volume, and health check. Configure the datasource in `auth-service/src/main/resources/application.properties` via environment variables. The existing `transaction-db` must not become a shared database.
3. **Create the user model.** Suggested `users(id UUID PK, email UNIQUE, password_hash, status, created_at, updated_at)` and `user_roles(user_id, role)` tables. Normalize email, validate input, store a password hash with Spring's `PasswordEncoder`, and make `CUSTOMER` the only self-registration role. Provision `MAKER`, `CHECKER`, and `ADMIN` through a controlled admin path or a local seed migration. Never store plaintext passwords. Spring's password storage guide recommends adaptive one-way encoders such as bcrypt or Argon2 and tuning their work factor.
4. **Connect Spring Security login to the user store.** Implement a `UserDetailsService` or authentication provider that loads the user, checks status, and maps roles to authorities. Configure the authorization-server security chain and a separate application/login chain following the Authorization Server getting-started guide. Keep CSRF protection enabled for the login/session and browser flows; do not copy a blanket `csrf.disable()` snippet into Auth Service. If `POST /auth/register` is a stateless API for Postman/mobile clients, exempt only that route from CSRF in a dedicated chain that does not use browser cookies, then apply validation and rate limits. If it uses a browser session, require a CSRF token.
5. **Persist OAuth2 clients and grants.** Use `JdbcRegisteredClientRepository`, `JdbcOAuth2AuthorizationService`, and `JdbcOAuth2AuthorizationConsentService` with their matching Spring Authorization Server schema as a starting point, or implement equivalent repositories. Do not ship in-memory clients/authorizations beyond a throwaway demo; a restart must not lose client or refresh-token state. Register a PKCE public client for Postman/browser practice and confidential clients for service calls. Store client secrets encoded, never in Git. The JPA persistence guide is an alternative to JDBC.

6. **Configure issuer, signing keys, and claims.** Choose one stable external issuer URL (for local use, for example `http://localhost:8081` if Auth Service listens there), configure `AuthorizationServerSettings`, load the signing key, and publish its public JWK. Customize **access tokens only** to add `aud=banking-api` and user `roles`; keep service tokens identified by their client and scopes. The token customizer reference shows the `OAuth2TokenCustomizer<JwtEncodingContext>` hook.
7. **Add local registration and administration APIs.** A first API can be `POST /auth/register` accepting only email and password, returning 201 and a user ID. `GET /auth/me` can return the authenticated user profile after Auth Service also validates that endpoint's bearer token or session. Role assignment, disablement, and password-reset endpoints require separate authorization, audit records, and tests. OAuth2 token and revocation endpoints are supplied by the framework; avoid parallel custom `/auth/login` and `/auth/refresh` implementations unless there is a specific client requirement.
8. **Add abuse controls and audit.** Rate limit registration, login, token, reset, and recovery endpoints. Use generic failure messages and avoid account enumeration. Record login success/failure, role changes, client changes, reset, revocation, and signing-key rotation without logging credentials or token values. Add MFA for staff roles before treating the system as anything beyond an exercise. OWASP authentication guidance covers generic errors, throttling, and MFA.

Example environment variables (values are examples, not committed secrets):

```
server.port=${SERVER_PORT:8081}
spring.datasource.url=${AUTH_DB_URL:jdbc:postgresql://localhost:5433/auth_db}
spring.datasource.username=${AUTH_DB_USER:auth_user}
spring.datasource.password=${AUTH_DB_PASSWORD}
app.security.issuer=${AUTH_ISSUER:http://localhost:8081}
app.security.signing-key-path=${AUTH_SIGNING_KEY_PATH}
```

If `auth-db` exposes port `5433` on the host, its Docker-internal URL is instead `jdbc:postgresql://auth-db:5432/auth_db`. Keep local development passwords in an ignored `.env` file or environment variables. Do not commit a sample with a usable private key or client secret.

Secure Gateway and each HTTP service

Add `spring-boot-starter-oauth2-resource-server` to the gateway and to each Servlet-based business service. Add `spring-cloud-starter-gateway-server-webflux` and the Spring Cloud **2025.0.x** BOM to `api-gateway`; that release train matches Spring Boot 3.5.x. A WebFlux gateway needs `SecurityWebFilterChain`, whereas the other starter services use Servlet `SecurityFilterChain`. See the Spring Cloud compatibility table and gateway security reference.

Use the same validation properties in the gateway and protected services, adjusting the issuer URL so each runtime can reach it while the configured issuer string still exactly matches the token's `iss`:

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=${AUTH_ISSUER}
spring.security.oauth2.resourceserver.jwt.audiences=banking-api
```

Where metadata/JWK lookup cannot use that public issuer URL from a container, also configure `spring.security.oauth2.resourceserver.jwt.jwk-set-uri` with the reachable internal JWK URL. Keep `issuer-uri` as well so issuer validation remains enabled. Do not blindly change issuer between `localhost` and a Docker hostname: the token and validator must agree. Use HTTPS endpoints in deployed environments.

Minimal Servlet security pattern for Account, Transaction, Approval, Ledger, Notification, and Reconciliation HTTP APIs:

```
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    return http
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

This is only the authentication baseline. Add route and method authorization in each service. With Spring Security's default JWT mapping, `scope=accounts.read` becomes `SCOPE_accounts.read`; a custom `roles` claim needs an explicit `JwtAuthenticationConverter` to become `ROLE_CUSTOMER`, etc. Do not write `hasRole("CHECKER")` until that converter exists. The default health exception should be narrowed to the endpoints needed by the runtime; secure all other actuator endpoints. For stateless bearer-only APIs, decide CSRF behavior explicitly based on whether any browser cookie/session credential is accepted. Keep CSRF enabled on Auth Service's login pages.

Gateway routes should expose only the intended Auth Service authorization endpoints and public business paths. A valid token at the gateway may be forwarded as the `Authorization` bearer header for phase 1, then validated again by the destination service. Strip incoming identity headers before forwarding. Configure rate limits at the gateway, but also apply tighter limits inside Auth Service for credential attacks. Do not expose `/internal/**` routes to clients.

At the gateway, allow unauthenticated access only to the exact registration and OAuth2/browser endpoints needed for the chosen flow: for example `POST /auth/register`, `GET /login`, the login form POST, `/oauth2/authorize`, `/oauth2/token`, the discovery document, and the public JWK Set endpoint. Auth Service still authenticates the user or OAuth2 client at those endpoints. Require a bearer token for business routes and do not open all of `/auth/**`. In Postman, configure **Authorization Code (with PKCE)** using Auth Service's `/oauth2/authorize` and `/oauth2/token`, the registered client ID, a registered callback URI, and an allowed scope. Exchange the code, then send the access token through the gateway as `Authorization: Bearer <token>`.

Suggested first implementation files (all under each service's existing `src/main` tree):

Service	Files to add
`auth-service`	`java/com/banking/authservice/config/AuthSecurityConfig.java`, `java/com/banking/authservice/config/AuthorizationServerConfig.java`, `java/com/banking/authservice/config/TokenClaimsConfig.java`, `java/com/banking/authservice/user/User.java`, `java/com/banking/authservice/user/UserRepository.java`, `java/com/banking/authservice/user/DatabaseUserDetailsService.java`, `java/com/banking/authservice/user/RegistrationController.java`, `resources/db/migration/V1__auth_schema.sql`, and environment-driven datasource settings
`api-gateway`	Gateway WebFlux security configuration, route definitions, JWT issuer/audience settings, and route-specific rate limits
`account-service`	Servlet resource-server configuration, JWT claims/authority mapping, account ownership policy, and method-level authorization tests

These are suggested file boundaries, not files already present in the checkout. Expand the same resource-server pattern as each remaining service gains endpoints.

Authorization rules to implement with the domain services

Action	Required identity and permission	Required service-side data check
Read an account or balance	`CUSTOMER` with `accounts.read`, or an explicitly authorized staff role	Compare token `sub` with stored `accounts.customer_id`; never compare with a caller-provided customer ID alone
Create a deposit/withdrawal/transfer	`CUSTOMER` with `transactions.create`, or a permitted maker workflow	Confirm ownership and status of every source account, currency/rules, and idempotency key before processing
Read a transaction	`transactions.read`	Check that the requester owns an involved account or has an authorized staff role
Create approval request	`MAKER` with `approvals.create`	Store maker's authenticated `sub`; enforce state and transaction association
Approve or reject	`CHECKER` with `approvals.decide`	Checker `sub` must differ from stored maker `sub`; atomically transition only a pending request; audit both identities
Assign roles or disable user	`ADMIN` with a dedicated administrative scope	Record actor, target, old/new state, reason, and time; do not let self-registration set staff roles

For a transfer, the gateway and Transaction Service authenticate the user; Transaction Service checks the source account belongs to that user. The internal call to Account Service must authenticate **Transaction Service as a service** with a narrowly scoped credential such as `accounts.execute`, and Account Service must authorize that caller and validate the operation contract. If Account Service needs the human actor for policy or audit, pass a verifiable delegated identity/token with the correct audience, or a signed internal assertion; do not accept an unsigned `actorId` as independent proof. Keep the existing transaction reference and idempotency rules described in `docs/transaction-service-reference.md` when that service is restored.

Kafka messages are not HTTP requests carrying a user bearer token. Restrict topic producers/consumers with broker credentials/ACLs and transport protection. Include event ID, transaction reference, and a captured actor ID for audit, but authorize a consumer's own operations through its service identity and validate event schema and origin. Deduplicate events as the transaction reference document requires.

Implementation order and acceptance checks

1. Add Auth DB, migrations, user/role model, password hashing, self-registration restricted to `CUSTOMER`, and controlled staff seeding.
2. Add Spring Authorization Server, persistent clients/grants, stable issuer and signing key, PKCE client, JWT access tokens, and a service client.
3. Add gateway routing and JWT validation, then resource-server validation to Account Service. Complete a real `register -> authorize/login -> token -> gateway -> account` path before adding more services.
4. Add account ownership checks. Next, secure Transaction and Approval Service as each is implemented, including maker/checker separation. Secure internal calls and Kafka access before end-to-end money movement.
5. Add logout/revocation behavior, refresh-token handling, abuse limits, audit, key rotation, and staff MFA. Then run integration tests with PostgreSQL and the gateway.

Minimum test matrix:

Test	Expected result
No token, malformed token, bad signature, expired token, wrong issuer, wrong audience	401 at gateway and at a directly called service
Valid token but missing scope/role	403 from the service; no state change
Customer A reads or transfers from Customer B's account	403 or a consistent 404 policy; no balance change
Maker tries to approve own request; checker tries to approve twice	Rejected; no duplicate transition
Public registration submits `ADMIN` or `CHECKER`	Request rejected or role ignored; new user remains `CUSTOMER`
Client sends fake `X-User-Id` or calls a service directly	Header ignored; service applies token and ownership rules
Auth Service unavailable after a token is issued	Already valid JWTs can be checked using cached public keys; new login/refresh fails safely
Logout or user disablement	Refresh/reissue stops; document and test the remaining access-token validity window
Restart Auth Service	Users, clients, grants, signing key, and valid refresh tokens survive

Use 401 for missing/invalid authentication and 403 for authenticated callers lacking permission. Test with a real JWT and HTTP boundary as well as unit tests of domain ownership checks. Do not mark the security work complete after a successful login alone.

References

- Project brief: `Banking_Microservices_Project (1).docx` in this repository.
- Existing historical transaction design: `docs/transaction-service-reference.md`.
- OWASP Microservices Security Cheat Sheet.
- Spring Authorization Server Reference.
- Spring Security 6.5 JWT Resource Server.
- Spring Cloud Gateway Security.